

Trong Node.js, cấu trúc phổ biến và hiệu quả nhất thường dựa trên **Kiến trúc đa tầng (Layered Architecture)** hoặc **MVC (Model-View-Controller)**.

Cây thư mục tiêu chuẩn

Plaintext

my-backend-project/

| — node_modules/ # Chứa các thư viện tải về từ NPM (tự động sinh ra)

| — src/ # Nơi chứa toàn bộ mã nguồn chính

| | — config/ # Cấu hình hệ thống (Database, API keys,

Environment...)

| | — controllers/ # Xử lý luồng dữ liệu (Nhận Request -> Trả Response)

| | — services/ # Xử lý "Core Business Logic" (Logic nghiệp vụ phức

tạp)

| | — models/ # Định nghĩa cấu trúc dữ liệu (Database Schemas)

| | — routes/ # Điều hướng URL đến đúng Controller

| | — middlewares/ # Các hàm đứng giữa chặn Request (Kiểm tra quyền,

báo lỗi...)

| | — utils/ # Chứa các hàm dùng chung (Helpers)

| | — app.js # File khởi tạo ứng dụng (Khai báo middlewares, routes

tổng)

| | — server.js # File chạy server chính (Lắng nghe port)

| — tests/ # Chứa các file viết test (Unit test, Integration test)

| — .env # Lưu các biến môi trường bảo mật (Database Password,

Secret Key)

| — .gitignore # Chứa tên các file/thư mục không muốn đẩy lên Github

| — package.json # "Chứng minh thư" của dự án (Khai báo tên, thư viện,

script chạy)

Chi tiết các thành phần quan trọng

Để bạn dễ hình dung, hãy tưởng tượng Backend của bạn là một **Nhà hàng**:

1. routes/ (Lễ tân - Người tiếp đón)

- **Nhiệm vụ:** Lắng nghe khách hàng gọi món gì (nhận URL và phương thức HTTP như GET, POST) và chỉ đường cho yêu cầu đó tới đúng người xử lý.
- *Ví dụ:* URL /api/users/login sẽ được routes điều hướng tới hàm login trong User Controller.

2. middlewares/ (Bảo vệ / Quản lý)

- **Nhiệm vụ:** Kiểm tra yêu cầu trước khi cho đi tiếp. Khách có mang theo thẻ thành viên không? Người này có bị cấm vào nhà hàng không?
- *Ví dụ:* Kiểm tra xem User đã đăng nhập chưa (Xác thực Token), kiểm tra dữ liệu gửi lên có hợp lệ không.

3. controllers/ (Người phục vụ / Bồi bàn)

- **Nhiệm vụ:** Nhận order từ routes, giao việc cho đầu bếp (services), lấy đồ ăn từ đầu bếp và bung ra cho khách (trả về JSON Response).
- **Lưu ý quan trọng:** Không nên viết logic tính toán phức tạp ở đây. Controller chỉ nên làm nhiệm vụ "Giao tiếp".

4. services/ (Đầu bếp - Trái tim của hệ thống)

- **Nhiệm vụ:** Nơi chứa toàn bộ **Business Logic** (Nghịệp vụ). Đầu bếp sẽ lấy nguyên liệu, nấu nướng, nêm nếm theo đúng công thức.
- *Ví dụ:* Logic tính toán giảm giá, kiểm tra mật khẩu, mã hóa dữ liệu trước khi lưu,... tất cả nằm ở đây. Việc tách biệt services và controllers giúp code gọn gàng và dễ tái sử dụng.

5. models/ (Kho nguyên liệu / Cấu trúc dữ liệu)

- **Nhiệm vụ:** Giao tiếp trực tiếp với Database (MySQL, MongoDB, PostgreSQL). Định nghĩa xem một bảng User sẽ có những cột nào (Tên, Tuổi, Email,...).

6. config/ (Sổ tay hướng dẫn của nhà hàng)

- **Nhiệm vụ:** Kết nối tới Database, kết nối tới dịch vụ gửi Email, các thiết lập về múi giờ...

7. utils/ hoặc helpers/ (Công cụ đa năng)

- **Nhiệm vụ:** Chứa các hàm nhỏ gọn, có thể dùng đi dùng lại ở nhiều nơi.
- *Ví dụ:* Hàm format ngày tháng (29/03/2026), hàm tạo mã ngẫu nhiên, hàm viết hoa chữ cái đầu...

Nguyên tắc vàng khi tổ chức code

- **Separation of Concerns (Tách biệt mối quan tâm):** File nào việc nấy.
- **Keep It DRY (Don't Repeat Yourself):** Nếu một đoạn code được dùng ở 2 nơi, hãy ném nó vào utils/ hoặc services/.